

Timing Blinders — Nebula 2026 Digital Track

RTL Timing Closure via Generative AI: Project Runbook

Shubhanshu Shivhare & Nagarjun BV

2026-08-02

Contents

1	0. About This Book	3
1.1	0.1 The Problem Statement	3
1.2	0.2 Canonical Pipeline Diagram (corrected, 2026-08-02)	4
1.3	0.3 Architecture Being Built	4
1.4	0.4 Two GenAI Entry Points (confirmed with Nagarjun’s flow diagram)	6
2	1. Pipeline Overview — Full Flow, Tools, Files, and Where the LLM Enters	7
2.1	1.1 The Full Pipeline	7
2.2	1.2 Build Status vs Plan	8
2.3	1.3 LLM Connection — Exact Mechanics (Stage 8, not yet built)	8
2.4	1.4 On the One-Time Nature of the Native-Build Work	8
3	2. Stage 0 — Digital Logic / STA Fundamentals Refresher	9
3.1	2.1 Goal	9
3.2	2.2 Design: <code>traffic_light.v</code>	9
3.3	2.3 Step-by-Step, By Hand	9
3.4	2.4 What “Reading the STA Report” Actually Means	10
3.5	2.5 The Violation Found and Fixed	11
3.6	2.6 Command Reference	11
3.7	2.7 Problems Hit + Fixes	11
4	3. Stage 1 — Generated Clocks + Multicycle Paths	12
4.1	3.1 Goal	12
4.2	3.2 Design: <code>clkdiv_mcp.v</code>	12
4.3	3.3 Bug #1 Hit + Fixed: X-Propagation in Simulation	12
4.4	3.4 Bug #2 Hit + Fixed: Generated Clock Declared at the Wrong Point	12
4.5	3.5 Final SDC (<code>rtl/stage1/clkdiv_mcp.sdc</code>)	13
4.6	3.6 Result	13
4.7	3.7 Command Reference	13
4.8	3.8 Problems Hit + Fixes Summary	14
5	4. Stage 6 Prep — SoC Integration & Clock Domain Crossing (CDC)	15
5.1	4.1 Why 5 Async Masters -> Minimum 10 Clock Domains	15
5.2	4.2 Why ~50,000 Standard Cells	15

5.3	4.3 What an SoC Actually Is, Structurally	15
5.4	4.4 CDC — The Core New Concept for Stage 6	16
5.5	4.5 Problems That Come Up at SoC Scale	16
5.6	4.6 Where This Fits in the Roadmap	17
6	5. Stage 5 — OpenROAD-flow-scripts (ORFS): Full RTL-to-GDS PPA Flow	18
6.1	5.1 Goal	18
6.2	5.2 The Blocker This Stage Started With, and Why Native Build Was Needed	18
6.3	5.3 Building Natively — Full One-Time Dependency Chain	18
6.4	5.4 Problems Hit + Fixes (in the order they appeared)	19
6.5	5.5 Running the Flow	21
6.6	5.6 What the Flow Actually Does, Stage by Stage	21
6.7	5.7 Real Results — gcd Design, sky130hd Platform	22
6.8	5.8 Command Reference	22
7	6. What’s Next	23

1 0. About This Book

This book is the single source of truth for the Timing Blinders project (Astera Labs Nebula 2026, Digital track — “Constraint Optimization through RTL Enhancement Using Generative AI”). It is written so that every stage — every command run, every bug hit, every fix applied, every number produced — can be reproduced by hand, from scratch, by someone who is still learning STA/VLSI concepts as they go.

Each chapter covers one stage of the project roadmap. Chapters follow the same structure: **Goal** → **Design/Setup** → **Step-by-step** → **Problems hit + fixes** → **Command reference**. Concepts introduced in one chapter (slack, WNS/TNS, generated clocks) are assumed known in later chapters — read in order the first time through.

1.1 0.1 The Problem Statement

Topic: “Constraint Optimization through RTL Enhancement Using Generative AI.”

Timing closure — making a chip design run at its target clock speed — is one of the most manual, iterative parts of chip design. Engineers analyze timing reports, find critical paths (signals that don’t arrive before the next clock edge), and manually rewrite RTL to fix them, then re-run synthesis and repeat.

Our job: build a system where an LLM does this instead — reads timing violations, proposes an RTL fix, and an automated toolchain verifies the fix actually helps AND doesn’t break functional correctness, before accepting it.

1.1.1 The four allowed RTL-fix techniques

- **Pipelining** — insert a register mid-path to split one slow cycle into two.
- **Logic restructuring** — rewrite logic to be functionally identical but with a shorter critical path.
- **Retiming** — move existing registers earlier/later without adding new ones.
- **FSM optimization** — re-encode state machine states (e.g. one-hot vs binary).

1.1.2 Required benchmark RTL

- 5 independent asynchronous master clock domains.
- =1 generated clock per master (10 domains minimum total).
- Clock domain crossings (CDC) between domains.
- Clock divider logic at multiple ratios.
- ~50,000 standard cells (realistic SoC-subsystem scale).

1.1.3 Official Nebula deliverables (scored against this checklist)

1. RTL timing analysis framework
2. GenAI-based RTL optimization engine
3. Critical path and timing violation analysis
4. Optimized RTL implementation
5. Timing, frequency and PPA comparison (before/after)

6. Formal equivalence verification report (proves optimized RTL == original)
7. Interactive demo showcasing the RTL optimization workflow

1.1.4 Official tools

Verilog/SystemVerilog, OpenSTA, Yosys, OpenROAD, SymbiYosys/EQY, Python, LLM/GenAI frameworks.

1.2 0.2 Canonical Pipeline Diagram (corrected, 2026-08-02)

This is the reviewed, corrected version of Nagarjun’s flow sketch — adds the per-edit EQY gate (not just an end-of-pipeline check), splits LINT-fail handling into deterministic-fix-first (token-saving) with GenAI only as fallback, adds explicit iteration caps on every loop, and names the critical-path JSON as GenAI’s actual input at the timing stage.

Why GenAI edits RTL only, never SDC (open question resolved 2026-08-02): some timing violations are actually constraint-file bugs, not logic-depth problems — e.g. Chapter 3’s `set_multicycle_path` fix (a legitimately multicycle path that had no exception declared) is exactly this class, fixed by editing the SDC, not the RTL. The pipeline deliberately does NOT let GenAI touch SDC for the graded loop, for two reasons: (1) the official Nebula problem statement lists exactly 4 RTL-fix techniques, all RTL-only; (2) EQY can formally prove an RTL edit preserves logic equivalence, but there is no equivalent formal proof that an SDC relaxation is “correct” versus just hiding the violation by loosening the deadline — a judge would read a loosened constraint as gaming the benchmark. Detecting/fixing genuine SDC bugs (missing multicycle exceptions, wrong input/output delay budgets) is flagged as a documented future-work item, not silently dropped — it would need its own, harder-to-build justification step (proving the relaxation matches real functional periodicity, the way Chapter 3’s `accept` signal justified its multicycle exception) before it could be safety-gated the way RTL edits are gated by EQY.

Why two EQY checks, not one: the per-edit gate (inside the timing loop) guards every individual GenAI edit before it’s allowed back into synthesis — this is the non-negotiable safety net. The final end-to-end check is a cheap extra sanity proof over the whole design right before committing to the expensive PPA flow; if it ever mismatches while the per-edit gate passed every edit, that indicates a bug in the loop itself (e.g. a state file got corrupted), not a bad RTL edit — treat it as halt-and-investigate, not retry-with-GenAI.

1.3 0.3 Architecture Being Built

```
run_sta(netlist, sdc)
    -> wraps OpenSTA, returns ranked critical paths + slack as
        structured JSON

propose_and_apply_edit(critical_path, rtl_snippet)
    -> the LLM call. Constrained to propose ONE of: pipelining, logic
        restructuring, retiming, FSM re-encoding. Emits edited RTL, not
        prose.

resynth_and_check(original.v, edited.v)
    -> wraps Yosys (re-synthesis), OpenSTA (re-run STA, compare slack),
        and EQY (formal equivalence gate). REJECTS the edit if EQY fails.
```

Timing Blinders — Canonical GenAI-in-the-loop RTL Timing-Closure Pipeline

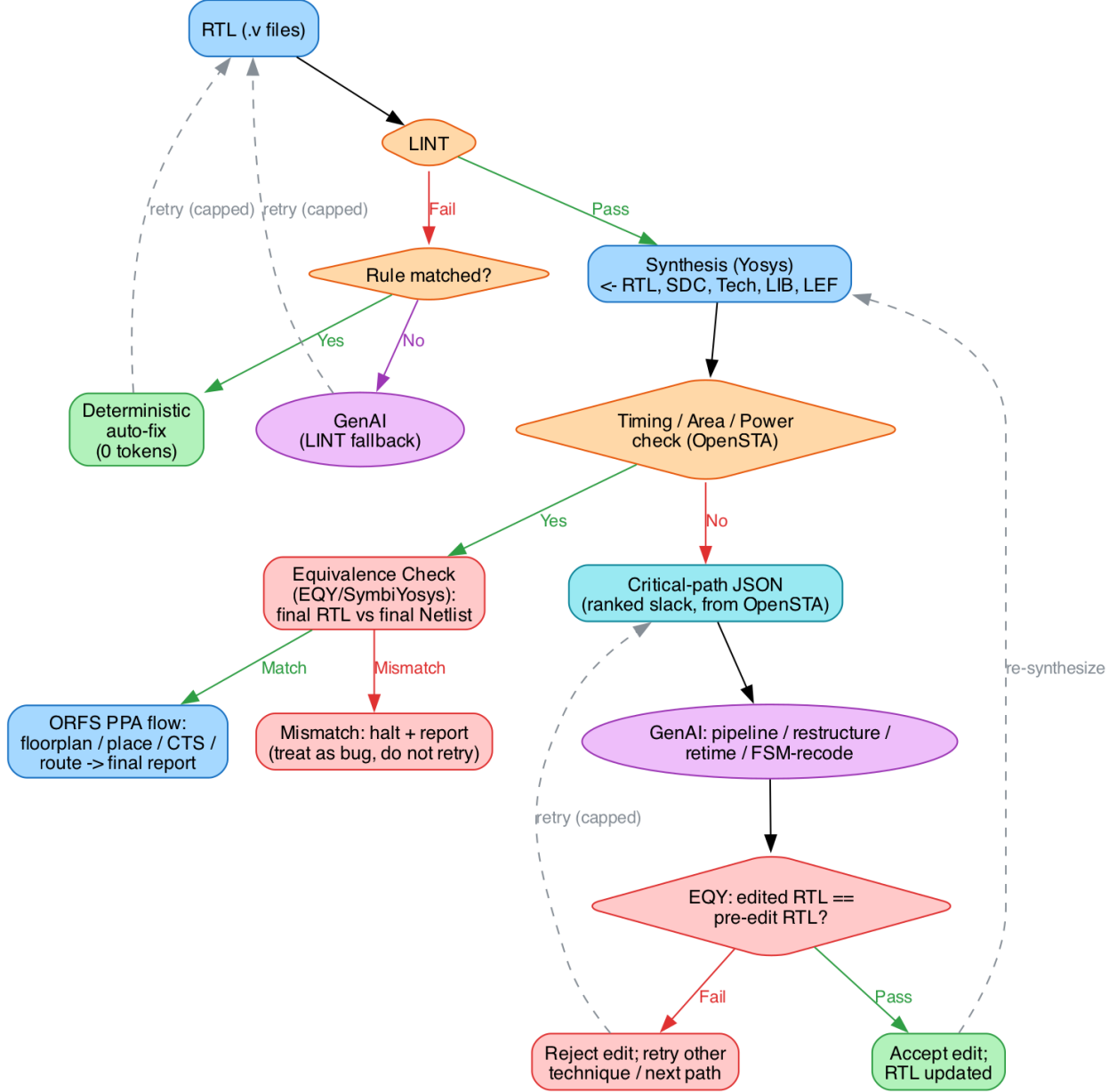


Figure 1: Canonical GenAI-in-the-loop pipeline

```
Loop: run_sta -> propose_and_apply_edit -> resynth_and_check ->
      (if improved AND equivalent) accept, else retry/move to next path
      -> repeat until timing closure or iteration budget exhausted.
```

Non-negotiable design principle: the EQY formal-equivalence gate must be built and working BEFORE the LLM loop is wired up. The LLM is never allowed to edit RTL without the safety net already in place.

1.4 0.4 Two GenAI Entry Points (confirmed with Nagarjun’s flow diagram)

The GenAI loop is not single-purpose — it engages at two distinct points in the pipeline, both feeding back into the same “Modify/suggest -> RTL” edge:

1. **LINT-fail loop** — RTL -> LINT -> (fail) -> auto-fix -> modify RTL. Revised after token-cost review (2026-08-02): most LINT failures (unused signal, blocking assignment in a sequential block, width mismatch) map to a fixed, mechanical rewrite — a deterministic rule-based fixer (plain Python/regex on the linter’s rule ID) handles these at **zero LLM tokens**. GenAI is only invoked as a fallback for LINT errors that don’t match a known deterministic rule (expected to be rare). This keeps the demo robust to messy input RTL without burning tokens on mechanical fixes — token budget is reserved for the timing-restructuring loop below, where real judgment is needed.
2. **Timing/Area/Power-fail loop** — Synthesis -> Timing/Area/Power check -> (fail) -> GenAI -> modify RTL. This is the core timing-closure loop described in the architecture above.

Open item flagged for the team: in Nagarjun’s diagram, the Equivalence Check (EQY) box sits only AFTER the Timing/Area/Power gate passes — i.e. it validates the *final* RTL against the *final* netlist once. Per this project’s non-negotiable design rule, EQY must instead validate **every individual GenAI-proposed edit** before that edit is allowed to re-enter synthesis — both from the LINT loop and the Timing/Area/Power loop — not just a single end-of-pipeline check. An iteration cap (max retries per critical path / per LINT error) should also be added to both GenAI loops to prevent an infinite modify-retry cycle if GenAI can’t converge on a passing edit.

2 1. Pipeline Overview — Full Flow, Tools, Files, and Where the LLM Enters

Read this before diving into individual stage chapters — it is the map of how everything connects end to end. Individual stage chapters go deep on one step at a time; this chapter shows how the steps chain together.

2.1 1.1 The Full Pipeline

1. RTL source (Verilog) `rtl/stageN/*.v`
|
2. Synthesis (Yosys) `yosys -p "read_verilog...; synth; ..."`
 `-> gate-level netlist` `*_synth.v`
 |
3. SDC constraints (hand-written) `rtl/stageN/*.sdc`
 |
4. STA (OpenSTA, via openroad -exit)
 `-> run_sta.tcl` script reads netlist+SDC, runs `report_checks`
 `-> structured JSON output` `artifacts/*.json` (ranked critical paths + slack)
 |
 ===== THIS JSON IS THE LLM'S INPUT =====
 |
5. LLM call (Claude API, Stage 8)
 Input: the JSON (critical path list) + the actual RTL snippet around the worst path (file, line range)
 Output: an EDITED RTL snippet, constrained to exactly one of 4 techniques (pipelining / restructuring / retiming / FSM re-encode) -- NOT prose, a tool_use structured code edit
 File: `propose_and_apply_edit.py` (Stage 8)
 |
6. Re-synthesis + re-STA (same Yosys/OpenSTA scripts as step 2-4, rerun on the edited RTL) -- did slack improve?
 |
7. Formal equivalence check (EQY/SymbiYosys, Stage 7)
 `.sby` config compares original RTL vs edited RTL: logically equivalent? NO -> reject edit, discard, try a different fix or move to the next critical path. YES -> accept edit, keep it.
 |
8. Loop steps 4-7 until timing closes (`WNS >= 0`) or iteration budget runs out
 |
9. Full PPA flow (OpenROAD via ORFS ``make``)
 `floorplan -> placement -> CTS -> routing -> final area/power/timing report`. File: `flow/designs/<platform>/<design>/config.mk`
 |
10. Python glue (Stage 9): one command runs steps 1-9 end-to-end + produces the demo (before/after RTL, before/after slack, EQY)

pass/fail trail)

2.2 1.2 Build Status vs Plan

Step	What it does	Status
1-4: RTL -> synth -> SDC -> STA -> JSON	Single-module hand-run flow	Built and working (Ch. 2/3)
5: LLM proposes edit	Claude API tool-use call	Not started (Stage 8)
6: Loop re-check	Re-run steps 2-4 on edited RTL	Not started (Stage 8)
7: EQY formal equivalence gate	Reject bad edits before acceptance	Not started (Stage 7) — must exist before Stage 8
9: Full PPA flow (ORFS)	floorplan/place/CTS/route + reports	Built and validated natively (Ch. 5)
10: End-to-end glue + demo	One command, full pipeline	Not started (Stage 9)

2.3 1.3 LLM Connection — Exact Mechanics (Stage 8, not yet built)

- **Trigger:** after step 4 produces the ranked critical-path JSON.
- **Input to the LLM:** that JSON (path, slack, involved cells/pins) plus a snippet of the actual `.v` source at the flagged lines.
- **Output from the LLM:** a `tool_use` block containing edited RTL text — not free-form chat prose — following Anthropic’s documented tool-use pattern (structured `tool_use` blocks, loop keyed on `stop_reason`).
- **File responsible:** `propose_and_apply_edit.py` (Anthropic Python SDK), called from the Stage 9 orchestration loop. Does not exist yet — comes only after the Stage 7 EQY gate is working, by explicit project design rule: never let the LLM edit RTL without the safety net already in place.

2.4 1.4 On the One-Time Nature of the Native-Build Work

Everything fixed while getting ORFS to build natively on macOS arm64 (CUDD, zstd, icu4c, boost::stacktrace, fmt header includes, Anaconda PATH contamination, the pinned-Yosys `-c` flag incompatibility with mainline Yosys — full detail in Chapter 5) is a ONE-TIME native compile of the toolchain binaries (`tools/install/OpenROAD/bin/openroad`, `tools/install/yosys/bin/yosys`). Once built, every future `make` run in `flow/` reuses these binaries directly — no rebuilding, no re-fixing. This would only need repeating if the `orfs/` repo is wiped or the project moves to a different machine.

3 2. Stage 0 — Digital Logic / STA Fundamentals Refresher

3.1 2.1 Goal

Hand-build one tiny design, hand-write its SDC, run it through synthesis + STA, find a real timing violation, and fix it by manually rewriting RTL (no tools/scripts doing the thinking) — to relearn STA vocabulary (slack, arrival time, required time, setup/hold, WNS/TNS) from first principles before any automation is layered on.

3.2 2.2 Design: `traffic_light.v`

Moore FSM: RED (4 cyc) -> GREEN (3 cyc) -> YELLOW (1 cyc) -> RED ... - **state/next_state**: 2-bit FSM state register + combinational next-state logic. - **count**: 3-bit datapath counter, counts cycles spent in current state. - Outputs **red/yellow/green**: one-hot decode of state.

File: `rtl/stage0/traffic_light.v`. Testbench: `rtl/stage0/traffic_light_tb.v`.

```
+-----+
|  traffic_light  |
clk -->|          |
rst_n->| state/next_state |--> red
| count (3-bit)    |--> yellow
|                  |--> green
+-----+
```

RED(4cyc) -> GREEN(3cyc) -> YELLOW(1cyc) -> RED ...

3.3 2.3 Step-by-Step, By Hand

3.3.1 2.3.1 Simulate functionally first (before touching timing)

```
cd rtl/stage0
iverilog -o sim/tb.out traffic_light.v traffic_light_tb.v
vvp sim/tb.out
```

Confirms functional correctness (FSM sequences RED->GREEN->YELLOW correctly) independent of timing — always do this before synthesis. A design that's functionally broken will also produce a nonsense STA report and waste your time chasing “timing bugs” that are actually logic bugs.

3.3.2 2.3.2 Synthesize to gates (sky130hd)

```
yosys -p "
read_verilog traffic_light.v
hierarchy -check -top traffic_light
proc; opt; fsm; opt; memory; opt
techmap; opt
dfflibmap -liberty <path to sky130_fd_sc_hd__tt_025C_1v80.lib>
abc -liberty <same liberty>
clean
write_verilog -noattr traffic_light_synth.v
```

```
stat -liberty <same liberty>
"
```

This is a full Yosys synthesis script: RTL -> generic gates (`proc/opt/fsm/memory`) -> tech-mapped flops (`dfflibmap`) -> tech-mapped combinational gates + optimization (`abc`) -> clean dangling wires -> write gate-level netlist. `stat -liberty` prints cell count + area — sanity-check this isn't zero/huge.

3.3.3 2.3.3 Write the SDC by hand

rtl/stage0/traffic_light.sdc:

```
create_clock -name clk -period 1.0 [get_ports clk]
set_input_delay -clock clk 1.0 [get_ports rst_n]
set_output_delay -clock clk 1.0 [get_ports {red yellow green}]
```

- `create_clock`: declares the clock, its period, which port it's on. Period is in **nanoseconds** by default in OpenSTA. 1.0ns period is deliberately aggressive/unrealistic for this design's logic depth, chosen ON PURPOSE to force a violation to practice on.
- `set_input_delay/set_output_delay`: tells STA how much of the clock period is “already spent” before/after this design's boundary (e.g. time for a signal to arrive from an upstream chip, or to be used downstream) — without these, STA assumes inputs arrive at time 0 and outputs have the whole period to work with, which is unrealistically generous.

3.3.4 2.3.4 Run OpenSTA

rtl/stage0/run_sta.tcl:

```
read_liberty <sky130hd typical-corner .lib>
read_verilog traffic_light_synth.v
link_design traffic_light
read_sdc traffic_light.sdc
report_checks -path_delay max -fields {slew cap input}
report_checks -path_delay min -fields {slew cap input}
report_wns
report_tns
exit
```

Run via the OpenSTA docker wrapper (`sta/sta.sh`, mounts repo root at `/data`):

```
./sta/sta.sh /data/rtl/stage0/run_sta.tcl
```

`report_checks -path_delay max` = setup checks (the usual “is my clock fast enough” check). `-path_delay min` = hold checks (data doesn't race ahead of the clock edge). Always check BOTH — a design can pass setup and fail hold or vice versa; they are independent failure modes.

3.4 2.4 What “Reading the STA Report” Actually Means

Key fields per path in `report_checks` output: - **Startpoint / Endpoint**: where the path begins (a flop Q, or an input port) and ends (a flop D, or an output port). - **data arrival time**: when the signal actually gets to the endpoint, accounting for every gate delay along the path. - **data required time**: the deadline — when it MUST arrive by (derived from the clock period + any

input/output delay budget set in the SDC). - **slack** = required — arrival. **Negative slack = violation**. Positive = margin (headroom before it would break). - **WNS** (Worst Negative Slack) = the single worst (most negative) slack across all paths — answers “how much do I need to speed up my *worst* path to close timing.” - **TNS** (Total Negative Slack) = sum of ALL negative slacks — tells you how widespread the problem is (one bad path vs. many).

3.5 2.5 The Violation Found and Fixed

First run: worst path was `_39_ -> red`, **slack -0.56ns**. Root cause: the original RTL had `assign red = (state == S_RED)` — i.e. the output was combinational logic sitting AFTER the state flop. That combinational decode gate delay was added on top of the flop’s clock-to-Q delay, pushing arrival time past the deadline.

The fix — retiming (one of the 4 allowed RTL-fix techniques): move the decode logic to evaluate off `next_state` and register the *result* directly (`red_r <= (next_state == S_RED)`), rather than registering `state` and then decoding combinationaly afterward. Functionally identical output sequence (since `state <= next_state` on the same edge anyway) — but now `red/green/yellow` drive straight off a flop’s Q pin with **zero** combinational gates in the path. See the `red_r/green_r/yellow_r` block in `traffic_light.v` (lines 52-62).

Re-run after fix: WNS improved -0.56ns -> -0.33ns. Worst path is now `rst_n -> _48_` (a different, `rst_n`-driven sync-reset-mux path at a flop’s D-input) — a separate, unaddressed issue (11 violations remain, up from 8, because loosening one constraint can expose others that were previously masked in `report_checks`’s default single-worst-path-per- endpoint view — don’t assume “violations went up” always means “got worse”; check WHICH paths and WHY). Confirms retiming can fully eliminate a specific critical path without touching functionality — this remaining `rst_n` issue is a different class of problem (looks like an input-delay budget issue on `rst_n`, not a logic-depth issue) and was deliberately left out-of-scope for this stage.

3.6 2.6 Command Reference

```
# functional sim
iverilog -o sim/tb.out traffic_light.v traffic_light_tb.v && vvp sim/tb.out

# synthesis (see full script above)
yosys -p "<script above>"

# STA
./sta/sta.sh /data/rtl/stage0/run_sta.tcl
```

3.7 2.7 Problems Hit + Fixes

None specific to Stage 0 beyond the intentional violation-and-fix exercise itself (that IS the exercise).

4 3. Stage 1 — Generated Clocks + Multicycle Paths

4.1 3.1 Goal

Real chips have more than one clock. This stage hand-builds the two SDC constructs needed for that: a **generated clock** (a clock derived from another clock inside the design, e.g. a divide-by-2) and a **multicycle path exception** (telling STA “this specific path is allowed more than 1 clock period to settle, because the surrounding logic only uses it every Nth cycle”).

4.2 3.2 Design: `clkdiv_mcp.v`

- `clk_div2`: divide-by-2 toggle flop off `clk` — this is the generated clock.
- `acc`: 32-bit accumulator clocked by `clk_div2`, fed by a deliberately deep combinational adder tree `((acc^operand)+(acc&operand)+(acc|operand)+operand)` that’s too slow to close in 1 `clk_div2` cycle at the given period — by design, so we have something to apply a multicycle exception to.
- `accept`: pulses once every 2 `clk_div2` cycles, which is the actual functional justification for the multicycle exception — the accumulator genuinely only needs a new result every other cycle.

```
clk --> [/2 toggle flop] --> clk_div2 --> [32-bit acc, deep adder tree]
                                     (needs 2 clk_div2 cycles)
```

4.3 3.3 Bug #1 Hit + Fixed: X-Propagation in Simulation

Original RTL had no initial value on `clk_div2` or `acc`. Trace through: `clk_div2`’s own always-block is `if (!rst_n) clk_div2<=0; else clk_div2<=~clk_div2;` — but if `clk_div2` starts as X, `~X` is still X forever, UNLESS the reset branch fires. The reset branch only fires on a **posedge** `clk` while `rst_n==0` — which it does — so `clk_div2` does reach a real 0. But `acc` is clocked on **posedge** `clk_div2`, and `clk_div2` never toggles (thus never **posedges**) while held at a stable 0 during the reset window, so `acc`’s own reset branch (`if(!rst_n) acc<=0`) never actually gets a clock edge to fire on before `rst_n` deasserts — leaving `acc` X forever via the `^` term in `sum_wide`.

Fix: added sim-only initial values (`reg clk_div2 = 1'b0;; reg acc = 32'd0;`) — does NOT change any real reset logic, just seeds the simulator’s initial state to something other than X, which real silicon doesn’t have (flops power up to a real 0 or 1, not X — X is a simulation artifact of “unknown”, not a hardware behavior). **Lesson:** any reg clocked by a derived/gated clock needs an explicit sim initial value, because you can’t always guarantee the derived clock will produce an edge during the reset window.

4.4 3.4 Bug #2 Hit + Fixed: Generated Clock Declared at the Wrong Point

First attempt: `create_generated_clock ... [get_ports clk_div2_out]` (targeting the output PORT). Result: `report_checks -path_group {clk clk_div2}` showed **zero paths** in the `clk_div2` group despite the design being fine.

Why: a generated clock only propagates FORWARD from the pin/port it is declared at. `clk_div2_out` is a port with nothing electrically downstream of it inside this design (it’s an output — its only “load” is outside the chip). Declaring the generated clock there means, from STA’s point of view, the clock signal reaches the boundary and stops — none of the accumulator’s flops (which are driven by the internal `clk_div2` net, not the port) are seen as being on that clock

domain at all. They're invisible to timing analysis — not violating, just silently unchecked, which is much worse than a reported violation.

Fix: find the actual toggle-flop's output pin in the SYNTHESIZED netlist (not the RTL — pin names only exist post-synthesis) and target that:

```
grep -n "clk_div2" clkdiv_mcp_synth.v | head
# find the cell instance driving the clk_div2 net, e.g. cell _528_,
# port .Q(clk_div2)

create_generated_clock -name clk_div2 -source [get_ports clk] \
    -divide_by 2 [get_pins _528_/Q]
```

Lesson: always declare a generated clock at its actual source pin (typically a toggle flop's Q), never at a downstream port — grep the synthesized netlist to find the real cell/pin name; it will NOT match any RTL signal name directly (_528_ bears no resemblance to clk_div2).

4.5 3.5 Final SDC (rtl/stage1/clkdiv_mcp.sdc)

```
create_clock -name clk -period 2.0 [get_ports clk]
create_generated_clock -name clk_div2 -source [get_ports clk] \
    -divide_by 2 [get_pins _528_/Q]
set_input_delay -clock clk_div2 0.3 [get_ports operand]
set_input_delay -clock clk_div2 0.3 [get_ports accept]
set_output_delay -clock clk_div2 0.3 [get_ports acc_out]
set_multicycle_path 2 -setup -from [get_clocks clk_div2] -to [get_clocks clk_div2]
set_multicycle_path 1 -hold -from [get_clocks clk_div2] -to [get_clocks clk_div2]
```

- -divide_by 2: tells STA the generated clock's period is 2x the master (so clk=2ns -> clk_div2=4ns automatically, no need to hand-specify).
- set_multicycle_path 2 -setup ...: extends the SETUP check's deadline from 1x period to 2x period for paths between clk_div2 and itself. This is the mechanism — without it, the exact same RTL would show a setup violation (deep adder can't finish in 4ns) even though functionally it's fine (accept only fires every 2 cycles).
- set_multicycle_path 1 -hold ...: **always pair a setup multicycle with an explicit hold multicycle** (usually staying at 1, i.e. explicitly NOT relaxed) — if you don't, some tools infer a matching hold relaxation automatically which can silently hide a real hold violation. Being explicit here is a deliberate defensive habit, not strictly required by OpenSTA in this exact case, but is standard industry practice.

4.6 3.6 Result

Final: WNS 0.00, TNS 0.00 — multicycle path closes cleanly at the relaxed 2-cycle (8ns) deadline; without the exception it would show roughly -3 to -4ns violation (adder needs ~7-8ns, only had 4ns budget).

4.7 3.7 Command Reference

```
# sim (after adding initial values)
iverilog -o sim/tb.out clkdiv_mcp.v clkdiv_mcp_tb.v && vvp sim/tb.out
```

```
# check final acc_out isn't X - should print e.g. final acc_out = 9c0b0230
```

```
# synth
```

```
yosys -p "read_verilog clkdiv_mcp.v; hierarchy -check -top clkdiv_mcp; \  
proc; opt; fsm; opt; memory; opt; techmap; opt; \  
dfflibmap -liberty <lib>; abc -liberty <lib>; clean; \  
write_verilog -noattr clkdiv_mcp_synth.v; stat -liberty <lib>"
```

```
# find the generated clock's real source pin
```

```
grep -n "clk_div2" clkdiv_mcp_synth.v | head
```

```
# STA
```

```
./sta/sta.sh /data/rtl/stage1/run_sta.tcl
```

4.8 3.8 Problems Hit + Fixes Summary

Problem	Root cause	Fix
acc_out simulates to all-X forever	derived clock never toggles during reset window, so accumulator's reset branch never gets a clock edge	add sim-only initial values to <code>clk_div2</code> and <code>acc</code> regs
clk_div2 timing group shows 0 paths	generated clock declared at output port (nothing downstream) instead of source pin	grep synthesized netlist for real toggle-flop cell, declare at <code>[get_pins <cell>/Q]</code>
report_clocks prints "invalid command"	wrong assumed command name — not a real OpenSTA command in this build	removed; confirmed real command surface via info commands

5 4. Stage 6 Prep — SoC Integration & Clock Domain Crossing (CDC)

Read this BEFORE writing any Stage 6 benchmark RTL. It explains the concepts needed to understand (not just copy) the 5-clock-domain, ~50k-cell benchmark design required by Nebula.

5.1 4.1 Why 5 Async Masters -> Minimum 10 Clock Domains

Nebula's benchmark spec requires: - 5 independent asynchronous master clocks. - ≥ 1 generated clock per master (e.g. divide-by-2/4 off that master, like the `clk_div2` exercise in Chapter 3).

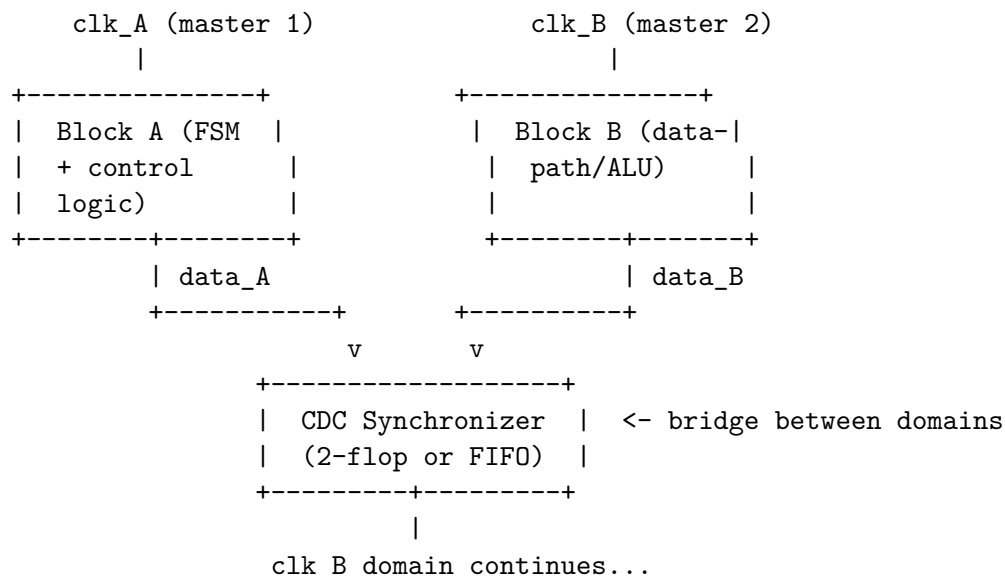
So: 5 masters + 5 generated (minimum 1 each) = **10 distinct clock domains minimum**. Each domain needs its own `create_clock` (masters) or `create_generated_clock` (derived clocks) declaration in the SDC — same mechanics as Chapters 2/3, just repeated per domain.

5.2 4.2 Why ~50,000 Standard Cells

Not a per-domain number — it's a TOTAL scale target across the whole design, meant to resemble a realistic SoC-subsystem rather than a toy module (Stage 0's `traffic_light` was on the order of 50-100 cells after synthesis). Reached not by writing one giant module, but by assembling several individually-simple blocks (FSMs, datapath/ALU logic, memory-like structures, CDC synchronizers, clock dividers) — each block is conceptually no harder than Chapter 2/3, the scale comes from combining enough of them.

5.3 4.3 What an SoC Actually Is, Structurally

Not one monolithic thing — several independently-clocked blocks, each individually as simple as a Stage 0 FSM or Stage 1 accumulator, wired together hierarchically (a top-level module instantiating child modules — ordinary Verilog hierarchy, nothing new). The only genuinely new concept versus Chapters 2/3 is what happens AT THE BOUNDARY where two different clock domains meet.



This pattern repeats across all 5 master domains + their generated clocks, with a synchronizer at every domain-boundary crossing.

5.4 4.4 CDC — The Core New Concept for Stage 6

The problem: when a signal generated in one clock's domain (say `clk_A` at 200MHz) is read by logic in a different, asynchronous clock's domain (`clk_B` at 150MHz), the two clocks' edges don't align predictably. Wiring signal A straight into a `clk_B`-clocked flop risks **metastability**: the receiving flop samples mid-transition and can output a value that's neither a clean 0 nor 1 for a brief period, which can then propagate downstream as corrupted data or, worse, resolve to different logic values in different downstream flops reading the same signal.

The standard fix — a synchronizer: typically 2 (sometimes 3) flops in series, clocked by the RECEIVING domain (`clk_B`), placed directly on the crossing signal. This gives any metastable state time to settle before the value is used by real logic. This is a mechanical, well-known circuit — not something to reinvent per-signal.

Multi-bit buses need more care: chaining a 2-flop synchronizer per bit independently does NOT work for multi-bit values — different bits can land in the receiving domain on different cycles, producing a garbage combination that was never a valid value in either domain. Standard fixes: - **Gray-coding** the value before crossing (only 1 bit changes per increment, so even a misaligned sample differs by at most 1 count) — common for crossing counters/pointers. - **Dual-clock FIFO** (asynchronous FIFO) — the standard block for crossing wide data buses, using gray-coded read/write pointers internally.

Stage 6 needs at least one instance of each: a simple 2-flop synchronizer for a single-bit control/status signal, and either a gray-coded pointer crossing or a small async FIFO for a multi-bit data crossing.

5.5 4.5 Problems That Come Up at SoC Scale

Problem	What it looks like	How it's resolved
Timing violation	<code>report_checks</code> shows negative slack on some path	Same 4 techniques as Chapter 2: pipelining, logic restructuring, retiming, FSM re-encoding — applied via the <code>run_sta -> propose_and_apply_edit -> resynth_and_check</code> loop (Stage 8), same idea as the Chapter 2 retiming fix, just automated
CDC violation	a signal crosses domains with no synchronizer, or a multi-bit bus crossed without gray-coding/FIFO	caught by a CDC-specific static check (industry tools: Questa CDC, Spyglass CDC; open-source: partial support in some Yosys passes) — fixed by inserting/correcting the synchronizer structure, not a timing fix

Problem	What it looks like	How it's resolved
Functional regression after a timing fix	an RTL edit that improves timing accidentally changes behavior	this is exactly what the EQY formal equivalence gate (Stage 7) exists to catch — every proposed edit is proven logically identical to the original before being accepted; if EQY fails, the edit is rejected regardless of how much it improved timing

5.6 4.6 Where This Fits in the Roadmap

This chapter is prep reading for **Stage 6** (build the benchmark RTL). It builds directly on Chapters 2-3 (slack, WNS/TNS, generated clocks, multicycle paths) — those single-domain fundamentals still apply to every individual block inside this SoC. CDC is the ADDITIONAL concept layered on top when multiple such blocks are connected together.

Stage 6 itself (actual RTL, actual SDC, actual synchronizer code, actual before/after numbers) gets its own chapter once built — this chapter is concepts-only, written before any Stage 6 code exists.

6 5. Stage 5 — OpenROAD-flow-scripts (ORFS): Full RTL-to-GDS PPA Flow

6.1 5.1 Goal

Run a design through the complete synthesis-to-layout flow (not just STA on a hand-synthesized netlist like Chapters 2/3) and get real PPA (Power/Performance/Area) numbers: floorplan, placement, clock tree synthesis (CTS), routing, and final area/power reports.

6.2 5.2 The Blocker This Stage Started With, and Why Native Build Was Needed

First attempt used ORFS’s official Docker image (`openroad/orfs:latest`). That image is **amd64-only** (confirmed via `docker manifest inspect` — no `arm64` manifest exists). Running it on Apple Silicon means Docker silently falls back to QEMU emulation, translating `amd64` instructions to `arm64` at runtime. The flow crashed with **SIGILL (illegal instruction) during CTS** — QEMU’s translation doesn’t correctly handle certain CPU-optimized instructions (`AVX2/BMI2`-class) that OpenROAD’s binaries use.

Fix chosen (confirmed with user over two alternatives — patching QEMU CPU flags, or deferring the issue): **build ORFS natively for arm64**. Runs real native Apple Silicon instructions, no emulation layer, no SIGILL risk.

Docker (amd64 image)	Native build (this chapter)
+-----+	+-----+
OpenROAD (amd64)	OpenROAD (arm64, native)
+-----+	+-----+
QEMU translation	direct execution
v	v
Apple Silicon (arm64) <-- SIGILL during CTS	Apple Silicon (arm64) <-- OK

6.3 5.3 Building Natively — Full One-Time Dependency Chain

This is a one-time compile. Once done, the binaries at `tools/install/OpenROAD/bin/openroad` and `tools/install/yosys/bin/yosys` are reused by every future `make` run — nothing here needs repeating unless the repo is wiped or moved to a new machine.

6.3.1 5.3.1 Homebrew dependencies

```
brew install or-tools qt@5 tcl-tk@8 swig libffi ruby python libomp \  
doxygen capnp bison flex spdlog zlib googletest yaml-cpp icu4c zstd  
brew install The-OpenROAD-Project/lemon-graph/lemon-graph
```

`qt@5` and `tcl-tk@8` are specifically the `VERSIONED` formulas — plain `qt/tcl-tk` are not sufficient. `lemon-graph` must come from OpenROAD’s own tap — the generic `brew install lemon` package doesn’t ship the CMake config files `find_package(LEMON)` needs.

6.3.2 5.3.2 CUDD (BDD package) — no Homebrew formula, build from source

```
git clone --depth=1 -b 3.0.0 https://github.com/The-OpenROAD-Project/cudd.git /tmp/cudd
cd /tmp/cudd
autoreconf
./configure --prefix=/Users/shubhanshu/Desktop/Nebula/orfs/dependencies
make -j10 install
```

6.3.3 5.3.3 Full git submodule fetch (shallow fetch fails on pinned commits)

```
git submodule deinit -f .
git submodule update --init --recursive # NOT --depth 1
```

Shallow (`--depth 1`) fails with “shallow file has changed since we read it” / commit not found, because the pinned submodule commit isn’t a branch tip GitHub will serve at shallow depth.

6.3.4 5.3.4 Anaconda contamination — must be scrubbed from every build invocation

If conda’s base environment auto-activates in your shell (check `CONDA_SHLVL/CONDA_PREFIX`), `/opt/anaconda3` ships its own `libfmt`, `Python3`, and `GTest` CMake configs that CMake can resolve ahead of Homebrew’s, causing version/ABI mismatches. Every build command in this chapter strips it:

```
CLEAN_PATH=$(echo "$PATH" | tr ':' '\n' | grep -v anaconda | tr '\n' ':' | sed 's/:$//')
env -u CONDA_PREFIX -u CONDA_SHLVL -u CONDA_DEFAULT_ENV -u CONDA_PROMPT_MODIFIER \
  PATH="$CLEAN_PATH" <build command>
```

6.3.5 5.3.5 The actual configure + build command

```
cd orfs
cmake -B tools/OpenROAD/build tools/OpenROAD \
  -D CUDD_DIR=$(pwd)/dependencies \
  -D TCL_LIBRARY=/opt/homebrew/opt/tcl-tk@8/lib/libtcl8.6.dylib \
  -D TCL_INCLUDE_PATH=/opt/homebrew/opt/tcl-tk@8/include \
  -D FLEX_INCLUDE_DIR=/opt/homebrew/opt/flex/include \
  -D CMAKE_PREFIX_PATH=/opt/homebrew \
  -D CMAKE_IGNORE_PATH="/opt/anaconda3/lib;/opt/anaconda3/include;/opt/anaconda3/bin" \
  -D Python3_FIND_STRATEGY=LOCATION \
  -D Python3_ROOT_DIR=/opt/homebrew/opt/python3 \
  -D CMAKE_EXE_LINKER_FLAGS="-L/opt/homebrew/lib -L/opt/homebrew/opt/icu4c@78/lib" \
  -D CMAKE_SHARED_LINKER_FLAGS="-L/opt/homebrew/lib -L/opt/homebrew/opt/icu4c@78/lib" \
  -D CMAKE_CXX_FLAGS="-DBOOST_STACKTRACE_GNU_SOURCE_NOT_REQUIRED" \
  -D CMAKE_INSTALL_PREFIX=$(pwd)/tools/install/OpenROAD

cmake --build tools/OpenROAD/build --target install -j 10
```

6.4 5.4 Problems Hit + Fixes (in the order they appeared)

#	Problem	Root cause	Fix
1	SIGILL during CTS	amd64 Docker image under QEMU translation on arm64	switch to native arm64 build
2	Shallow submodule fetch fails	pinned commit not servable at shallow depth	full git submodule update --init --recursive
3	brew --prefix or-tools fails	not installed	brew install or-tools
4	qt@5 not found	plain qt insufficient, needs versioned formula	brew install qt@5
5	tcl-tk@8 not found	same, versioned formula required	brew install tcl-tk@8
6	swig missing	not installed	brew install swig
7	find_package(LEMON) fails	generic lemon formula lacks CMake config	brew install The-OpenROAD-Project/lemon-graph
8	CUDD_LIB NOTFOUND	no Homebrew formula for CUDD exists at all	build from source into orfs/dependencies, pass CUDD_DIR
9	FLEX_INCLUDE_DIR/TCL_LIBRARY NOTFOUND	CMake couldn't auto-locate versioned Homebrew paths	pass explicit -D paths
10	fmt::v11::vformat link error (fft_test) + Python3/GTest resolving from /opt/anaconda3	conda base env auto-activated, shadowing Homebrew	scrub /opt/anaconda3 from PATH, unset CONDA_*, pin CMAKE_PREFIX_PATH=/opt/homebrew + CMAKE_IGNORE_PATH
11	no member named 'format' in namespace 'fmt' in AbstractFlowAnalysis.npps (persisted after fix #10)	genuine upstream vendoring bug: vendored fmt 11.2.1 npps <fmt/format.h> for fmt::format(), 10 slang source files only included <fmt/core.h>	patched all 10 files: #include <fmt/core.h> -> #include <fmt/format.h>
12	ld: library 'zstd' not found	bare -lzstd linker flag with no -L path to Homebrew's lib dir	add -L/opt/homebrew/lib via CMAKE_EXE_LINKER_FLAGS
13	ld: library 'icudata' not found	icu4c is keg-only, not symlinked into /opt/homebrew/lib	add -L/opt/homebrew/opt/icu4c@78/lib

#	Problem	Root cause	Fix
14	<code>Boost.Stacktrace</code> requires <code>_Unwind_Backtrace...</code> <code>Define _GNU_SOURCE</code>	<code>boost::stacktrace</code> 's glibc-only guard doesn't recognize macOS's libunwind-based availability	<code>-DBOOST_STACKTRACE_GNU_SOURCE_NO</code> in <code>CMAKE_CXX_FLAGS</code>
15	<code>yosys -c script.tcl</code> : "Option 'c' does not exist"	tried reusing OSS CAD Suite's mainline Yosys (0.67), which dropped <code>-c</code> ; ORFS's own pinned/patched Yosys submodule still has it	build ORFS's own <code>tools/yosys</code> submodule separately, same fix set (<code>CMAKE_PREFIX_PATH</code> , <code>CMAKE_IGNORE_PATH</code> , linker <code>-L</code> paths, <code>BOOST_STACKTRACE_GNU_SOURCE_NOT</code>
16	GDS export step: <code>Error: KLayout not found</code>	KLayout (GDS/DRC/LVS viewer+tool) not installed	non-blocking — only affects GDS/DRC/LVS targets, not synthesis/STA/PPA numbers; install KLayout separately if GDS output is needed later

6.5 5.5 Running the Flow

```
cd orfs/flow
```

```
make DESIGN_CONFIG=./designs/skyl30hd/gcd/config.mk YOSYS_EXE=<path to ORFS's own yosys>
```

`YOSYS_EXE/OPENROAD_EXE` resolve automatically to `tools/install/{yosys,OpenROAD}/bin/...` if not found on `PATH` (see `flow/scripts/variables.mk`), so this only needs to be set explicitly if a different Yosys happens to be on `PATH` first (as OSS CAD Suite's was in this case).

6.6 5.6 What the Flow Actually Does, Stage by Stage

```
1_synth      -- Yosys: RTL -> generic gates -> tech-mapped netlist (same
               idea as Chapter 2's manual synth script, run via ORFS's
               own scripts)
2_floorplan  -- define chip area, utilization, power grid (PDN)
3_place      -- global placement (approximate positions) then detailed
               placement (legalized, no overlaps)
4_cts        -- Clock Tree Synthesis: insert clock buffers to distribute
               the clock signal to every flop with minimal skew
5_route      -- global routing (coarse) then detailed routing (DRC-clean
               wires)
6_finish     -- final reports: timing (WNS/TNS), area, power, IR drop
```

Each stage writes reports to `flow/reports/<platform>/<design>/base/`, named `N_<stage>.rpt` (or `_final/_check` variants), matching the numbering above — read the report matching the number of the stage of interest, e.g. `4_cts_final.rpt` for the post-CTS timing snapshot.

6.7 5.7 Real Results — gcd Design, sky130hd Platform

Constraint (`designs/sky130hd/gcd/constraint.sdc`): `clk_period = 1.1ns` (deliberately aggressive, same “force a violation on purpose” idea as Chapter 2’s traffic light).

Stage	WNS (max)	TNS (max)
Post-CTS (<code>4_cts_final.rpt</code>)	-1.39 ns	-60.39 ns
Final, post-route (<code>6_finish.rpt</code>)	-1.47 ns	-65.72 ns

Post-CTS clock skew: **0.01 ns setup skew** — tight, expected for a small design with a straightforward clock tree.

Final area/power (`6_finish.rpt` cell report): - **Design area:** 4977 μm^2 at **85% utilization** - **Total power:** 8.56 mW (8.56e-03 W) - **Worst-case IR drop:** 1.26 mV on VDD (0.07%), 0.876 mV on VSS (0.05%) — both comfortably small relative to the 1.8V supply - **Cell breakdown:** 839 total cells — 226 multi-input combinational, 171 timing-repair buffers (inserted by the resizer to fix timing/DRC), 35 sequential (flops), 75 tap cells, 7 clock buffers, 319 fill cells

Reading this honestly: WNS is still negative (-1.47ns) — this design does NOT close timing at this aggressive 1.1ns period as-is. That’s expected and fine for validating the flow itself; timing closure via RTL edits is exactly what Stage 8’s LLM loop will attempt later, on the Stage 6 benchmark design, using this same report format as its input.

6.8 5.8 Command Reference

```
# one-time native build (see full dependency list above)
cmake -B tools/OpenROAD/build tools/OpenROAD <all -D flags above>
cmake --build tools/OpenROAD/build --target install -j 10
cmake -B tools/yosys/build tools/yosys -DCMAKE_BUILD_TYPE=Release \
  -DYOSYS_SKIP_ABC_SUBMODULE_CHECK=ON -DCMAKE_INSTALL_PREFIX=$(pwd)/tools/install/yosys \
  <same CMAKE_PREFIX_PATH/CMAKE_IGNORE_PATH/linker/boost flags as OpenROAD build>
cmake --build tools/yosys/build --target install -j 10

# every future run, reusing the built binaries
cd orfs/flow
make DESIGN_CONFIG=./designs/sky130hd/gcd/config.mk

# read results
cat reports/sky130hd/gcd/base/6_finish.rpt      # final WNS/TNS/area/power
cat reports/sky130hd/gcd/base/4_cts_final.rpt  # post-CTS timing + skew
```

7 6. What's Next

Stage	Topic	Status
2	TCL fundamentals for Yosys/OpenSTA/EQY scripting	Planned
3	Yosys synthesis scripting (<code>run_synth.py</code> wrapper)	Planned
4	OpenSTA JSON scripting (<code>run_sta.py</code> , the LLM's input format)	Planned
6	Full 5-domain/50k-cell benchmark RTL, actual code+SDC	Planned
7	SymbiYosys/EQY formal equivalence gate	Planned — must precede Stage 8
8	Claude API tool-use agentic loop	Planned
9	Python orchestration / end-to-end demo	Planned

Each future stage gets its own chapter appended to this book, following the same Goal -> Setup -> Step-by-step -> Problems+Fixes -> Command Reference structure used throughout.